GUIÓN ACADÉMICO: REGRESIÓN LINEAL EN JULIA

(GLM - temp_w - Pluto)

🎯 INTRODUCCIÓN GENERAL (antes de comenzar)

Idea base que debes dominar: La regresión lineal es el algoritmo más fundamental del aprendizaje automático supervisado. Su objetivo es encontrar la **línea recta** (o plano, en dimensiones mayores) que mejor describe la relación entre una variable de entrada (x) y una variable de salida (y). "Mejor" significa: aquella que minimiza el error cuadrático entre lo que el modelo predice y los valores reales.

Lo que vamos a presentar: cómo se construye paso a paso —desde cero, sin librerías— en Julia, y comparado al final con la versión profesional usando GLM.

📻 DIAPOSITIVA 2 — LA ANALOGÍA DE LA RADIO

Qué decir:

> "Para entender los componentes de la regresión lineal voy a usar la analogía de una radio vieja que intenta sintonizar una canción. Cada perilla de la radio representa un elemento matemático del algoritmo."

Mapeo completo:

- **x, y (ondas y canción):** (x) son los datos de entrada (las ondas invisibles que llegan a la antena), (y) es la meta, lo que queremos reproducir (la canción original).
- **w (perilla principal) = peso o pendiente:** define qué tan inclinada es la recta. Es el procesamiento central de la señal.
- **b (ajuste fino) = sesgo o intercepto:** desplaza la recta hacia arriba o abajo. Limpia el ruido de fondo.
- **f(x) (la bocina) = la predicción:** lo que el modelo entrega como resultado actual.
- **J (la estática) = la función de costo:** mide qué tan lejos está la predicción de la realidad.
- **α (alfa) = tasa de aprendizaje:** qué tan fuerte o brusco giras las perillas en cada ajuste.

🔥 Posibles preguntas y cómo responder:

P: ¿Por qué usas una analogía en lugar de irte directo a las matemáticas? R: Porque los parámetros w y b son abstractos. La analogía de la radio hace tangible que w controla la "forma"

de la señal (la inclinación) y b la corrige (desplazamiento), mientras que α regula la agresividad del ajuste.

P: ¿Qué pasa si alfa es muy grande o muy pequeña? R: Si α es muy grande, la mano gira la perilla con demasiada fuerza y se pasa la estación (diverge). Si es muy pequeña, tarda una eternidad en sintonizar (convergencia lenta). Esto se verá en los casos 3 y 4.

P: ¿w y b son lo mismo que la pendiente e intercepto de la recta escolar y = mx + b? R: Exactamente. Solo cambia la notación: en machine learning se usa `w` (weight) en lugar de `m`, pero es matemáticamente idéntico.

📐 DIAPOSITIVA 3 — EL MODELO

Qué decir:

> "La fórmula matemática del modelo es f(w,b)(x) = w·x + b. Es la ecuación de una recta donde w es la inclinación y b es el corte en el eje y. En Julia, la traducción a código es literal y directa."

```julia
function modelo(w, b, x)
    return w * x + b
end
```

🔥 Posibles preguntas:

P: ¿Por qué se le llama "regresión lineal" si solo es una recta? R: "Lineal" se refiere a que los parámetros (w y b) aparecen elevados a la primera potencia. Si tuviéramos varias entradas, seguiría siendo lineal: $f = w_1x_1 + w_2x_2 + ... + b$. La recta es solo el caso más simple con una sola entrada.

P: ¿Por qué Julia y no Python? R: Julia está diseñado específicamente para cómputo científico. Tiene sintaxis matemática natural, compilación JIT (Just-In-Time) que lo hace casi tan rápido como C, y maneja tipos numéricos de forma muy eficiente. La fórmula matemática y el código se ven prácticamente iguales.

P: ¿Qué significa "JIT"? R: Just-In-Time compilation. Julia compila el código a lenguaje máquina justo antes de ejecutarlo, logrando velocidades cercanas a C/Fortran sin perder la facilidad de un lenguaje interpretado como Python.

P: ¿Qué pasaría si en lugar de w·x + b tuviéramos $w \cdot x^2 + b$? R: Ya no sería regresión lineal sino polinómica. La geometría del problema cambiaría: la superficie de costo seguiría siendo convexa respecto a los parámetros, pero el modelo se curvaría.

💰 DIAPOSITIVA 4 — FUNCIÓN DE COSTO

Qué decir:

> "La función de costo J(w,b) mide el error total del modelo. Es el promedio de los errores al cuadrado entre las predicciones y los valores reales."

Fórmula: $J(w,b) = (1/2m) \Sigma (f(w,b)(x^i) - y^i)^2$

Explicación a fondo:

- **El cuadrado** cumple dos funciones:
  1. Penaliza errores grandes más severamente (un error de 4 pesa 16, no 4).
  2. Evita que errores positivos y negativos se cancelen entre sí.
- **El 1/2m:** el `m` es el número de datos (promedio). El `1/2` es un truco matemático: cuando derivamos la función, el exponente 2 baja y se cancela con el 1/2, dejando la fórmula limpia.

```julia
function funcion_costo(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i])^2 for i in 1:m)
    return suma / (2.0 * m)
end
```

Nota importante: en Julia, los bucles explícitos no son lentos como en Python, gracias al compilador JIT especializado.

🔥 Posibles preguntas:

P: ¿Por qué no usar el valor absoluto |error| en lugar del cuadrado? R: El valor absoluto no es diferenciable en cero, y las derivadas son el corazón del algoritmo de aprendizaje. El cuadrado es suave y diferenciable en todo punto, lo que hace el cálculo matemático mucho más elegante. Además, penaliza más los errores grandes, que suelen ser los peores para un modelo.

P: ¿Qué pasa si J(w,b) = 0? R: Significa que el modelo predice perfectamente los datos de entrenamiento. En la práctica casi nunca ocurre (habría sobreajuste u *overfitting*), y si pasa con muchos datos reales probablemente hay un error.

P: ¿Qué es MSE? R: Mean Squared Error (Error Cuadrático Medio). Es exactamente esta función de costo, y es la más común en regresión.

P: ¿Por qué `1:m` y no `0:m-1` como en Python? R: Julia indexa arrays desde 1 por convención matemática, no desde 0 como Python o C. Esto es más natural para matemáticos.

📊 DIAPOSITIVA 5 — DERIVADAS PARCIALES

Qué decir:

> "Para saber en qué dirección mover w y b y reducir el costo, necesitamos las derivadas parciales de J respecto a cada parámetro. Son como la voz de la conciencia matemática que nos dice hacia dónde girar cada perilla."

Fórmulas:

- $\partial J/\partial w = (1/m)\, \Sigma\, (\hat{y} - y) \cdot x$
- $\partial J/\partial b = (1/m)\, \Sigma\, (\hat{y} - y)$

Donde $\hat{y}$ es la predicción y y es el valor real.

```julia
function dj_dw(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i]) * x_datos[i] for i in 1:m)
    return suma / m
end

function dj_db(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum(modelo(w, b, x_datos[i]) - y_datos[i] for i in 1:m)
    return suma / m
end
```

🔥 Posibles preguntas:

P: ¿Qué significa geométricamente una derivada parcial? R: Es la pendiente de la función de costo en una dirección específica. $\partial J/\partial w$ dice: "si muevo solo w un poquito, ¿cuánto cambia J?". Es el vector que apunta hacia dónde crece el error; nosotros nos movemos en dirección contraria.

P: ¿Por qué la derivada respecto a b no lleva el x? R: Porque cuando derivas $(wx + b - y)^2$ respecto a b, el x desaparece por regla de la cadena. Respecto a w, el x se queda multiplicando porque es el "coeficiente" de w dentro del paréntesis.

P: ¿Qué es el gradiente? R: El vector formado por todas las derivadas parciales: $\nabla J = [\partial J/\partial w, \partial J/\partial b]$. Apunta en la dirección de **máximo crecimiento** de J. Por eso vamos en la dirección contraria

para minimizar.

P: ¿Y si tuviera 100 parámetros? R: Tendrías 100 derivadas parciales (una por parámetro). El principio es idéntico; solo cambia la escala. En la práctica se usa notación vectorial/matricial para no escribir cada una.

🎛️ DIAPOSITIVA 6 — GRADIENTE DESCENDENTE (girando perillas al unísono)

Qué decir:

> "El gradiente descendente es el algoritmo que ajusta w y b iterativamente. La regla teórica es: nuevo_w = w − α · ∂J/∂w. Pero hay un detalle crítico: las actualizaciones deben ser **simultáneas**."

Regla teórica:
$w \leftarrow w - \alpha \cdot (\partial J/\partial w)$
$b \leftarrow b - \alpha \cdot (\partial J/\partial b)$

El error común:

```julia
w = w - alfa * grad_w    # ❌ Si actualizamos w aquí...
b = b - alfa * grad_b    # ...el nuevo w contamina el cálculo de b
```

La solución correcta (swap con variables temporales):

```julia
temp_w = w - alfa * grad_w
temp_b = b - alfa * grad_b
w = temp_w
b = temp_b
```

Así garantizamos un solo paso unificado hacia el óptimo, en lugar de dos pasos desincronizados.

🔥 Posibles preguntas:

P: ¿Qué pasa realmente si no hago la actualización simultánea? R: El algoritmo sigue funcionando en muchos casos, pero deja de ser estrictamente gradiente descendente (sería más parecido a Gauss-Seidel). Puede converger a resultados ligeramente distintos y el comportamiento teórico ya no se garantiza.

P: ¿Por qué se resta (w − α·grad) en lugar de sumar? R: Porque el gradiente apunta hacia el máximo. Queremos ir al mínimo, por eso restamos (vamos en la dirección opuesta). Si

sumáramos, sería "gradiente ascendente" (útil, por ejemplo, para maximizar algo como una recompensa).

P: ¿Cómo sé cuándo detener las iteraciones? R: Tres criterios comunes: (1) número fijo de iteraciones (300, 1000), (2) cuando J cambie menos que un umbral ε, o (3) cuando el gradiente sea casi cero. En la práctica se combinan.

P: ¿Es el gradiente descendente el único método? R: No. Existen variantes como *Stochastic Gradient Descent*, *Adam*, *RMSprop*, y métodos de segundo orden como Newton. Para regresión lineal incluso existe una solución cerrada (ecuaciones normales) que da la respuesta exacta sin iterar. Pero el gradiente descendente es el fundamento conceptual de todo el deep learning moderno.

🎛️ DIAPOSITIVA 7 — TABLERO DE CONTROL (lectura de gráficas)

Qué decir:

> "Al entrenar el modelo, tenemos cuatro visualizaciones simultáneas que actúan como tablero de control:"

1. **Regresión Lineal:** muestra los puntos originales (datos reales) y la recta predictiva final.
2. **Evolución:** un "abanico" de rectas a través del tiempo; muestra cómo la recta va aprendiendo iteración tras iteración, desde una recta aleatoria inicial hasta la óptima.
3. **Superficie de Costo:** la parábola J(w). Es el terreno físico que recorre el algoritmo buscando el mínimo.
4. **Convergencia:** el costo J vs el número de iteraciones. Es el "reloj" del algoritmo: muestra qué tan rápido aprende.

🔥 Posibles preguntas:

P: ¿Por qué la superficie de costo es una parábola? R: Porque J es cuadrática en los parámetros (tiene un término al cuadrado). En regresión lineal, **siempre** es convexa —como un tazón—, lo que garantiza que hay un único mínimo global y el gradiente descendente siempre lo encuentra (si α es razonable).

P: ¿Qué significan los puntos naranjas en la superficie? R: Cada punto es una iteración del algoritmo. Muestra el camino que recorre w (o w,b) en su viaje hacia el mínimo (el diamante verde).

⛰️ DIAPOSITIVA 8 — GEOMETRÍA Y ESPACIO

Qué decir:

> "Hay que distinguir dos gráficas que se parecen pero dicen cosas distintas:"

PARÁBOLA — J(w) vs w:

- Representa el "mapa del terreno" matemático.
- Muestra **por qué** funciona el algoritmo: la superficie convexa garantiza un único mínimo global.

CONVERGENCIA — J(w,b) vs iteraciones:

- Muestra la dimensión temporal.
- Indica **qué tan rápido** desaparece el error.
- Avisa cuándo el modelo ya terminó de aprender (cuando se aplana).

🔥 **Posibles preguntas:**

P: ¿Siempre la convergencia tiene forma de "L" (cayendo y aplanándose)? R: En un caso ideal, sí: cae exponencialmente y se aplana. Si no se aplana, el modelo no ha terminado. Si sube, el α es demasiado grande (diverge). Si cae lento sin aplanarse, falta paciencia (más iteraciones).

P: ¿Puede una función de costo tener varios mínimos? R: En regresión lineal NO, es siempre convexa. En redes neuronales SÍ, y ahí aparece el problema de los mínimos locales, por eso hacen falta técnicas más sofisticadas.

🎯 DIAPOSITIVA 9 — CASO 1: CONVERGENCIA IDEAL

Qué decir:

> "Primer caso: 3 puntos, α = 0.01, 300 iteraciones. Es el escenario ideal."

Observaciones:

- La curva de convergencia decae exponencialmente en las primeras 50 iteraciones y luego se aplana.
- Los puntos naranjas en la superficie de costo aterrizan casi exactamente en el diamante verde (mínimo).
- Resultado: w* = 1.6378, j* = 0.011607.
- **Analogía:** la mano gira la perilla con pulso firme, y la estática desaparece limpiamente.

🔥 **Posibles preguntas:**

P: ¿Por qué solo 3 puntos? R: Es un caso didáctico. Con 3 puntos es fácil visualizar y los cálculos son rápidos. En la vida real se usan miles o millones.

P: ¿Por qué α = 0.01 y no otro valor? R: Es un valor clásico de inicio. Balancea rapidez con estabilidad. Los tres factores (α, escala de datos, número de iteraciones) deben calibrarse juntos.

P: ¿Cómo sé que este es "el mejor" modelo posible para esos datos? R: Porque J es convexa en regresión lineal: el mínimo alcanzado es el mínimo global. No hay un w,b mejor que ese.

🎚️ DIAPOSITIVA 10 — CASO 2: AJUSTE FINO

Qué decir:

> "Segundo caso: 7 puntos, pero esta vez los valores de x son más grandes (van hasta 7). Esto cambia la geometría."

El reto: cuando x es grande, el gradiente (que multiplica por x) se vuelve grande también, y los pasos del algoritmo se vuelven gigantes. Con α = 0.01 el modelo diverge.

La solución: reducir α a 0.003 para compensar la escala. Los pasos son más pequeños y densos, pero el aterrizaje es más limpio.

Resultado: w* = 1.1406, j* = 0.041011.

🔥 **Posibles preguntas:**

P: ¿Por qué cambió el α al cambiar el rango de x? R: Porque en la derivada ∂J/∂w aparece multiplicado por x. Si x crece, el gradiente crece, y α·gradiente se vuelve enorme. Hay que compensar reduciendo α.

P: ¿Hay forma de no tener que ajustar α cada vez? R: Sí: **normalizar los datos** (escalarlos entre 0 y 1 o estandarizarlos a media 0 y varianza 1). Es práctica estándar en machine learning y vuelve al modelo mucho más estable.

P: ¿Qué es "feature scaling"? R: Exactamente eso: ajustar la escala de las variables de entrada. Reduce el problema de elegir α y acelera el entrenamiento.

💥 DIAPOSITIVA 11 — CASO 3: ERROR O DIVERGENCIA

Qué decir:

> "Tercer caso: 7 puntos, α = 0.2, solo 50 iteraciones. Aquí el modelo explota."

Fallo matemático: α sobrepasa el umbral teórico de estabilidad. Cada salto rebota más lejos del mínimo en lugar de acercarse.

Colapso numérico: el error explota a magnitudes de 10^{51}. La recta "se va para abajo" en la gráfica.

Analogía: la mano es tan brusca que pasa la estación de largo, rebota violentamente al otro lado del espectro, y nunca encuentra la frecuencia correcta.

🔥 **Posibles preguntas:**

P: ¿Qué es NaN y cuándo aparece? R: Not a Number. Aparece cuando los números se vuelven tan grandes que desbordan la precisión del computador (típicamente al superar 10^{308} en float64). Es la señal inequívoca de que el modelo divergió.

P: ¿Cómo prevenir esto? R: Tres estrategias: (1) bajar α, (2) normalizar datos, (3) implementar *gradient clipping* (recortar el gradiente si supera un umbral).

P: ¿Existe un α "máximo teórico"? R: Sí, para regresión lineal el umbral de estabilidad es $\alpha < 2/L$ donde L es la constante de Lipschitz (relacionada con el mayor valor propio de la matriz $X^T X$). En la práctica uno experimenta empíricamente.

⏳ DIAPOSITIVA 12 — CASO 4: FALTA DE PACIENCIA

Qué decir:

> "Cuarto caso: 5 puntos, α = 0.005, solo 8 iteraciones. El problema ya no es α, es tiempo."

Síntomas visuales: la curva de convergencia sigue cayendo sin aplanarse. El diamante verde queda atascado a la mitad de la ladera con w = 0.50 y un costo alto (4.02).

Lección: apagar la máquina justo cuando la estática empezaba a disiparse. El modelo iba bien, pero no se le dio tiempo.

🔥 **Posibles preguntas:**

P: ¿Cómo saber el número correcto de iteraciones? R: Observando la curva de convergencia: cuando se aplana, el modelo terminó. Mejor aún: implementar *early stopping* con un conjunto de validación.

P: ¿Es mejor siempre usar muchas iteraciones para estar seguros? R: No. Demasiadas iteraciones pueden causar sobreajuste (overfitting) en modelos más complejos, y además gastan tiempo de cómputo. Hay un punto óptimo.

P: ¿Cuál es la diferencia entre este caso y el caso 3? R: El caso 3 falla por inestabilidad (α muy grande, diverge). El caso 4 falla por impaciencia (α adecuado pero iteraciones insuficientes, converge pero aún no llegó). Ambos tienen alto costo, pero por razones opuestas.

🏁 DIAPOSITIVA 13 — CONCLUSIÓN

Qué decir:

> "Al final tenemos dos caminos para resolver el mismo problema:"

LIBRERÍAS (GLM):

```julia
using GLM
lm(@formula(y ~ x), df)
```

- **Ventajas:** eficiente, robusto, probado.
- **Desventajas:** aceptas ciegamente la tolerancia, la estabilidad numérica y el algoritmo interno. Es una caja negra validada por terceros.

CÓDIGO MANUAL:

```julia
temp_w = w - alfa * grad_w
```

- **Ventajas:** completamente transparente. Cada variable es una decisión consciente. Entiendes cada componente.
- **Desventajas:** frágil, expuesto a colapsos numéricos como NaN si no se cuida.

Mensaje final:

> "El gradiente descendente es literalmente rodar hacia abajo por una superficie matemática. Es el esqueleto que sostiene todo el aprendizaje automático moderno, desde la regresión lineal hasta las redes neuronales gigantes."

🔥 Posibles preguntas finales:

P: ¿Qué es GLM? R: Generalized Linear Models, un paquete de Julia que implementa regresión lineal, logística, y otros modelos lineales generalizados, con soluciones cerradas optimizadas y estadísticas (p-valores, intervalos de confianza).

P: ¿Por qué entonces uno haría esto a mano si GLM ya existe? R: Por tres razones: (1) pedagogía —entender lo que pasa dentro de la caja negra—, (2) control total sobre el proceso, y (3) porque cuando uno pasa a redes neuronales, ya no hay solución cerrada: toca implementar gradiente descendente sí o sí.

P: ¿Cuál es la relación entre esto y deep learning? R: Es el fundamento directo. Una red neuronal es, esencialmente, muchas regresiones lineales apiladas con funciones de activación no lineales en medio. El gradiente descendente (y su pariente *backpropagation*) es el mismo algoritmo, solo que aplicado a millones de parámetros.

P: ¿Qué es Pluto? R: Pluto.jl es un entorno de notebooks reactivos para Julia (similar a Jupyter, pero con la particularidad de que cuando cambias una celda, todas las celdas que dependen de ella se recalculan automáticamente). Ideal para visualizaciones interactivas como las que se muestran en los casos 1 a 4.

P: ¿Qué diferencia hay entre regresión lineal simple y múltiple? R: La simple tiene UNA variable de entrada ($f = wx + b$). La múltiple tiene VARIAS ($f = w_1x_1 + w_2x_2 + ... + b$). El algoritmo es idéntico, solo cambia la dimensión del vector w.

🧠 PREGUNTAS TRAMPA O PROFUNDAS QUE PODRÍAN HACERTE

P: ¿La regresión lineal sirve para predecir solo o también para entender? R: Ambas. Los coeficientes w (ponderados) dicen cuánto influye cada variable en y. Es interpretable, a diferencia de modelos más complejos como redes neuronales que son cajas negras.

P: ¿Qué supuestos matemáticos tiene la regresión lineal? R: Idealmente asume: (1) linealidad entre x e y, (2) errores con distribución normal, (3) homocedasticidad (varianza constante del error), (4) independencia de los errores. Si se violan, el modelo sigue funcionando pero las predicciones y los intervalos de confianza pueden ser malos.

P: ¿Cómo se evalúa si un modelo de regresión lineal es "bueno"? R: Con métricas como R^2 (coeficiente de determinación: cuánta varianza explica el modelo), RMSE (raíz del error cuadrático medio), o MAE (error absoluto medio). Idealmente se evalúa en datos no vistos (test set).

P: ¿Qué pasa si mis datos no son lineales? R: Se pueden transformar (por ejemplo, aplicar logaritmo) o usar modelos más flexibles: regresión polinómica, splines, árboles de decisión, redes neuronales, etc.

P: ¿Se puede hacer regresión lineal sin gradiente descendente? R: Sí. La solución cerrada es: $w = (X^T \cdot X)^{-1} \cdot X^T \cdot y$ (ecuaciones normales). Es exacta y sin iteraciones, pero requiere invertir una matriz, lo cual es caro computacionalmente para datasets muy grandes.

P: ¿Por qué entonces usamos gradiente descendente si hay solución exacta? R: Porque la solución exacta requiere invertir una matriz de tamaño (nºfeatures × nºfeatures), lo cual es $O(n^3)$. Para cientos de miles de parámetros, es imposible. El gradiente descendente escala mucho mejor.

✅ CHECKLIST FINAL ANTES DE PRESENTAR

- [ ] Entiendo qué son w, b y α y puedo explicarlos sin leer.
- [ ] Puedo escribir la fórmula de J(w,b) y explicar por qué el cuadrado y por qué el 1/2.
- [ ] Sé la diferencia entre gradiente descendente y descenso por coordenadas (actualización simultánea vs secuencial).
- [ ] Reconozco los tres fallos típicos: divergencia (α alto), lentitud (α bajo), atasco (iteraciones pocas).
- [ ] Puedo explicar por qué la superficie de costo es convexa en regresión lineal.
- [ ] Puedo comparar GLM vs código manual y defender ambas posturas.

Tip de cierre: Si te hacen una pregunta que no sepas responder, no inventes. Di algo como: *"Excelente pregunta. Lo que sí puedo decir es [lo que sabes]; la parte que mencionas no la trabajé a detalle pero sería un buen punto para profundizar."* Eso demuestra madurez académica.